

This is the consolidated **Master Development Plan & Software Requirement Specification (SRS)** for the **Unidentifiable Vehicle Tracking & Profiling (UVTP)** module.

This document serves as the "Blue-Print" for your implementation. It finalizes the logic, architecture, and data structures before you write the final production code.

Master Development Plan: UVTP Module

Project: Smart Road Vision System (Final Year Project)

Module: Unidentifiable Vehicle Tracking & Profiling (UVTP)

Objective: To detect, track, and profile vehicles that lack identifiable license plates, generating actionable intelligence for traffic enforcement.

1. System Architecture & Data Flow

The system operates on a **Pipeline Architecture**. Data flows linearly from the Edge (Camera) to the Cloud (Server), passing through distinct processing blocks.

The Processing Pipeline:

1. **Ingestion:** ESP32-CAM streams MJPEG video to Python Server.
2. **Primary Inference (YOLOv10):** Detects Vehicle and License_Plate objects.
3. **Anomaly Logic Gate:** Checks if a valid plate exists within the vehicle's bounding box.
4. **Deep Tracking (DeepSORT + FastReID):** Assigns a unique Session_ID to the vehicle based on visual features (not plates).
5. **Profiling (Attribute Net):** Extracts Color, Make, and Type from the best-quality frame.
6. **Reporting:** JSON payload sent to the Police Dashboard.

2. Software Requirement Specifications (SRS)

2.1 Functional Requirements (FR)

FR-01: The Anomaly Trigger (Logic Gate)

- **Definition:** The system must differentiate between a "Temporary Occlusion" and a "Missing Plate."

- **Logic:** A vehicle is flagged as "Unidentifiable" IF:
 - Object Class is Car, Truck, or Bike.
 - **AND** No License_Plate bounding box is detected within the Vehicle ROI (Region of Interest) for **>15 consecutive frames** (approx. 0.5 - 1.0 seconds).
 - **OR** A License_Plate is detected, but its pixel area is < 500px (too small/far) or OCR confidence is < 40%.

FR-02: Robust Visual Tracking (Vehicle-ReID)

- **Definition:** The system must maintain the identity of the unidentifiable vehicle across the entire field of view.
- **Constraint:** Standard DeepSORT (Person ReID) is forbidden.
- **Implementation:** Must use **FastReID** with a **ResNet50** backbone trained on **Veri-776**.
- **Metric:** The Cosine Distance between the vehicle's embedding in Frame \$N\$ and Frame \$N+1\$ must be < 0.2 to confirm identity.

FR-03: Multi-View Snapshot Heuristics

- **Definition:** The system must intelligently select and save exactly three (3) evidence images per tracked vehicle.
- **Heuristic Logic:**
 1. **Img_Entry:** First frame where the vehicle is fully visible.
 2. **Img_Best:** The frame with the **Maximum Bounding Box Area** (closest to camera) **AND Highest Laplacian Variance** (least blurry).
 3. **Img_Exit:** The last frame before the vehicle leaves the ROI.

FR-04: Deep Attribute Profiling

- **Definition:** The system must generate a text description from the **Img_Best**.
- **Attributes Required:**
 - **Color:** (Red, Blue, White, Black, Silver, Grey, Yellow).
 - **Type:** (Sedan, SUV, Hatchback, Rickshaw, Bike).
 - **Orientation:** (Front-View, Rear-View, Side-View) - *Derived from tracking vector.*

FR-05: Session Management

- **Definition:** Each unidentifiable vehicle constitutes a "Session."
- **ID Format:** UID-{TIMESTAMP}-{CAMERA_ID}-{INCREMENT} (e.g., UID-20240204-CAM01-0045).
- **Lifecycle:** Session opens on trigger -> Updates on tracking -> Closes on exit -> Data flushes to DB.

2.2 Non-Functional Requirements (NFR)

- **Latency:** The tracking loop must run at **minimum 15 FPS** on the GPU server to ensure no frame skipping. Attribute extraction (Profiling) can be asynchronous (running in a

separate thread).

- **Accuracy:** Vehicle Re-ID accuracy (Rank-1) must exceed **85%** (vehicles must not swap IDs when passing each other).
- **Storage:** Snapshot images must be compressed (JPG, 80% quality) to save disk space.

3. Detailed Implementation Logic

3.1 The "Ghost" Logic (Python Pseudo-Code)

This is the core algorithm determining *when* to track a vehicle as a violator.

Python

```
class TrackedObject:
    def __init__(self, id):
        self.id = id
        self.plate_seen_count = 0
        self.total_frames_seen = 0
        self.is_ghost = False # "Ghost" means unidentifiable
        self.snapshots = {'entry': None, 'best': None, 'exit': None}

def logic_loop(frame, detections, tracker):
    # 1. Update Tracker
    tracks = tracker.update(detections, frame)

    for track in tracks:
        # 2. Check for Plate INSIDE the Vehicle Box
        has_plate = check_intersection(track.bbox, plate_detections)

        track.total_frames_seen += 1
        if has_plate:
            track.plate_seen_count += 1

        # 3. The "Ghost" Threshold
        # If we've seen the car for 20 frames, but seen the plate in less than 5...
        if track.total_frames_seen > 20 and track.plate_seen_count < 5:
            track.is_ghost = True
            initiate_profiling(track, frame)
```

3.2 The Tech Stack Specifications

- **Detection Model:** YOLOv10-Medium (Fine-tuned on IDD + Pakistani Traffic).
 - **Tracker:** DeepSORT (modified).
 - **Feature Extractor (The "Brain"):**
 - **Library:** FastReID.
 - **Architecture:** ResNet50-ibn (Instance Batch Normalization is crucial for handling lighting changes on roads).
 - **Weights:** Veri-776 Pre-trained.
 - **Input Size:** 256x256 pixels.
 - **Attribute Classifier:** HydraPlus-Net or a simple Multi-Head CNN (Head 1: Color, Head 2: Type).
-

4. Data Schema (Output)

This is the JSON structure your system will generate for every "Ghost Vehicle."

JSON

```
{
  "event_id": "UID-20240204-CAM01-0045",
  "timestamp": "2024-02-04T14:30:00Z",
  "camera_location": "University Road, Karachi",
  "violation_type": "UNIDENTIFIABLE_VEHICLE",
  "vehicle_profile": {
    "predicted_color": "White",
    "predicted_make": "Toyota",
    "predicted_type": "Sedan",
    "confidence_score": 0.92
  },
  "evidence": {
    "snapshot_entry_url": "/evidence/2024/02/UID-0045_entry.jpg",
    "snapshot_best_url": "/evidence/2024/02/UID-0045_best.jpg",
    "snapshot_exit_url": "/evidence/2024/02/UID-0045_exit.jpg"
  },
  "tracking_metadata": {
    "duration_in_frame": "4.5s",
    "direction": "North-bound"
  }
}
```

}

5. Execution Roadmap

Since we have the "Finer Details" locked, here is the immediate execution order:

1. **Step 1: The "Brain" Swap (Critical Path)**
 - Download FastReID and Veri-776 weights.
 - Replace deep_sort/feature_extractor.py with the Vehicle-ReID code I provided earlier.
 - *Verification:* Run a test video. Ensure cars maintain IDs even when they turn.
2. **Step 2: The Logic Gate**
 - Implement the check_intersection function (checking if a Plate BBox is inside a Vehicle BBox).
 - Tune the frames_threshold (Start with 15 frames).
3. **Step 3: The Profiler**
 - Train a small classifier (or use a pre-trained one from HuggingFace) for "Vehicle Color."
 - Connect it to the Img_Best output.

This plan is now **Frozen**. No further "research" is needed. You are ready to code Step 1.

Shall I provide the specific command to download the "Veri-776" weights now?